Entropy Maps: Space-Optimal Implementation of Bernoulli Maps through Information-Theoretic Coding

Alexander Towell
atowell@siue.edu

July 29, 2025

Abstract

We present entropy maps as a space-optimal implementation technique for Bernoulli maps—functions that provide noisy observations of latent mathematical mappings. The key insight is that by using prefix-free codes and cryptographic hash functions, we can achieve information-theoretically optimal space usage while maintaining the latent/observed distinction fundamental to Bernoulli types. An entropy map observes a latent function $f : X \to Y$ by hashing domain elements to prefix-free codes representing codomain values, with the code lengths determined by the desired observation distribution. This enables oblivious computation where the stored representation reveals nothing about the latent function beyond what can be inferred from queries. We demonstrate applications to privacy-preserving set membership, approximate function representation, and space-efficient probabilistic data structures. The framework reveals that optimal space usage requires accepting observation errors—a fundamental trade-off between fidelity and efficiency in observing latent functions.

1 Introduction

The Bernoulli framework distinguishes between latent mathematical objects and their computational observations. While previous work has focused on the algebraic properties of these observations, a critical question remains: how can we *implement* observations space-efficiently?

This paper introduces *entropy maps* as an answer. An entropy map implements observations of a latent function $f : X \to Y$ using space proportional to the entropy of the observation distribution—achieving information-theoretic optimality.

1.1 The Implementation Challenge

Consider implementing an observed set membership function $\widetilde{\mathbf{1}_A} : X \to \texttt{Bool}$ where $A \subseteq X$. The latent function $\mathbf{1}_A$ returns `true` for $x \in A$ and `false` otherwise. Traditional approaches either:

- Store A explicitly, using $O(|A| \log |X|)$ bits
- Use a Bloom filter, achieving $O(|A|)$ bits but with false positives

Entropy maps provide a principled framework: the space required is exactly the entropy of the observation distribution, achieving optimal compression while making the latent/observed distinction explicit.

1.2 Core Insight: Hashing to Prefix-Free Codes

The key idea is deceptively simple:

1. Assign prefix-free codes to codomain values based on desired observation probabilities
2. Use a cryptographic hash function to map domain elements to binary strings
3. Check if the hash prefix matches any code to determine the observed output

By carefully choosing code lengths, we can implement any observation distribution while using minimal space.

2 Theoretical Framework

2.1 Prefix-Free Codes and Observation Probabilities

Definition 2.1 (Prefix-Free Code Assignment)**.** *Given a codomain Y and desired observation probabilities $\{p_y\}_{y \in Y}$, a prefix-free code assignment is a mapping $c : Y \to \{0,1\}^*$ such that:*

1. *No code is a prefix of another: $\forall y, y' \in Y, y \neq y' \implies c(y) \not\sqsubseteq c(y')$*
2. *The probability of observing y under uniform hashing equals p_y*

Theorem 2.2 (Kraft-McMillan Inequality for Observations)**.** *A probability distribution $\{p_y\}_{y \in Y}$ can be realized by prefix-free codes if and only if:*

$$\sum_{y \in Y} p_y \leq 1 \tag{1}$$

The equality holds for complete observations; inequality allows for undefined outputs.

2.2 From Latent Functions to Observed Functions

Definition 2.3 (Entropy Map)**.** *Given a latent function $f : X \to Y$ and a cryptographic hash function $h : X \times \mathbb{N} \to \{0,1\}^\omega$, an entropy map with seed s implements the observed function:*

$$\widetilde{f}_s(x) = \begin{cases} y & \text{if } \exists y \in Y : c(y) \sqsubseteq h(x,s) \\ \perp & \text{otherwise} \end{cases} \tag{2}$$

where $\sqsubseteq$ denotes the prefix relation.

The critical insight is that we don't store the function directly—we store only a seed that, combined with the hash function, produces the desired observations.

2.3 Optimal Code Length Selection

Theorem 2.4 (Optimal Code Lengths)**.** *For a latent function $f : X \to Y$ with distribution $\mathbb{P}[f(x) = y \mid x \sim \mathcal{U}(X)] = q_y$, the space-optimal code lengths to achieve observation probabilities p_y are:*

$$\ell_y = -\log_2 p_y \tag{3}$$

This achieves expected code length (entropy):

$$\mathcal{H} = -\sum_{y \in Y} p_y \log_2 p_y \tag{4}$$

3 Latent/Observed Duality in Entropy Maps

3.1 Set Membership as Function Observation

Consider the set membership problem through the latent/observed lens:

Example 3.1 (Bernoulli Set Membership)**.** *For a latent set $A \subseteq X$, we want to observe membership queries through $\widetilde{\mathbf{1}_A}$. Design goals:*

- *No false negatives:* $\mathbb{P}\left[\widetilde{\mathbf{1}_A}(x) = \texttt{true} \mid \textit{latent } x \in A\right] = 1$
- *Controlled false positives:* $\mathbb{P}\left[\widetilde{\mathbf{1}_A}(x) = \texttt{true} \mid \textit{latent } x \notin A\right] = \alpha$

Solution: Assign codes such that:

- $c(\texttt{true})$ *has total probability* α *(e.g., all strings starting with* $\underbrace{00\ldots0}_{k \text{ bits}}$ *for* $k = -\log_2 \alpha$*)*
- $c(\texttt{false})$ *has probability* $1 - \alpha$
- *Find seed* s *where* $\forall x \in A : c(\texttt{true}) \sqsubseteq h(x, s)$

This reveals the fundamental nature of Bloom filters: they implement entropy maps for set membership with specific code assignments.

3.2 The Oblivious Property

Definition 3.2 (Oblivious Observation)**.** *An observed function $\widetilde{f}$ is k-oblivious if the stored representation reveals at most k bits of information about the latent function f.*

Theorem 3.3 (Entropy Maps are Maximally Oblivious)**.** *An entropy map storing only a k-bit seed is k-oblivious, achieving the optimal trade-off between space and observation fidelity.*

This property is crucial for privacy-preserving applications: the stored representation reveals minimal information about the latent function.

4 Construction Algorithms

4.1 Finding Suitable Seeds

The core challenge is finding a seed s that maps all elements correctly:

Input: Latent function $f : X \to Y$, codes $c : Y \to \{0,1\}^*$
Output: Seed s implementing $\widetilde{f}$
repeat
  $s \leftarrow$ random seed;
  valid $\leftarrow$ `true`;
  for $x \in dom(f)$ **do**
    if $\neg\exists y : c(y) \sqsubseteq h(x, s) \wedge y = f(x)$ **then**
      valid $\leftarrow$ `false`;
      break;
    end
  end
until *all* $x \in dom(f)$ *map correctly*;
return s;

Algorithm 1: Entropy Map Construction

Theorem 4.1 (Expected Construction Time)**.** *For $n = |dom(f)|$ elements mapped to codes of average length ℓ, the expected number of trials is:*

$$\mathbb{E}[trials] = 2^{n\ell} \tag{5}$$

4.2 Multi-Level Construction

For large domains, we can use hierarchical construction:

Input: Domain partition $X = X_1 \cup \ldots \cup X_k$, functions $f_i : X_i \to Y$
Output: Seeds $s_1, \ldots, s_k$ and router function
for $i = 1$ *to* k **do**
  $s_i \leftarrow$ ConstructEntropyMap(f_i, c);
end
Router $r : X \to [k]$ mapping x to its partition;
return $(s_1, \ldots, s_k, r)$;

Algorithm 2: Hierarchical Entropy Map

5 Applications

5.1 Privacy-Preserving Membership Tests

Entropy maps enable membership tests that reveal nothing beyond query results:

Example 5.1 (Private Set Intersection)**.** *Two parties holding sets A and B can compute $|A \cap B|$ without revealing their sets:*

1. *Party 1 constructs entropy map for $\widetilde{\mathbf{1}_A}$ with false positive rate α*
2. *Party 1 sends only the seed s (e.g., 256 bits)*
3. *Party 2 queries all $x \in B$ and counts matches*
4. *Expected intersection size: $\frac{matches - \alpha|B|}{1-\alpha}$*

5.2 Approximate Function Storage

Example 5.2 (Compressed Function Tables)**.** *Store a function $f : \{0,1\}^{32} \to \{0,1\}^8$ (4GB uncompressed) approximately:*

- *For frequently accessed inputs, ensure correct outputs*
- *For rare inputs, allow errors with rate α*
- *Storage: $O(\text{frequent inputs} \times 8 + \log(1/\alpha))$ bits*

5.3 Probabilistic Caching

Example 5.3 (Space-Efficient Cache)**.** *Implement a cache admission policy:*

- *Latent: "Should item x be cached?"*
- *Observed: Entropy map with tunable false positive rate*
- *Benefit: Constant space regardless of item universe size*

6 Information-Theoretic Analysis

6.1 Rate-Distortion Trade-off

Theorem 6.1 (Fundamental Space-Accuracy Trade-off)**.** *For any implementation of observations of a function $f : X \to Y$ with error rate ϵ, the minimum space required is:*

$$S \geq |dom(f)| \cdot [H(Y) - H(\epsilon)] \tag{6}$$

where $H(Y)$ is the entropy of the output distribution and $H(\epsilon)$ is the binary entropy of the error rate.

Entropy maps achieve this bound, confirming their optimality.

6.2 Observation Channel Capacity

Viewing entropy maps as communication channels:

Definition 6.2 (Observation Channel)**.** *The observation channel induced by an entropy map has:*

- *Input alphabet: X (domain)*
- *Output alphabet: $Y \cup \{\perp\}$ (codomain plus undefined)*
- *Channel matrix: Determined by code assignments and hash function*

Theorem 6.3 (Channel Capacity)**.** *The capacity of an entropy map observation channel is:*

$$C = \max_{p(x)} I(X; \widetilde{Y}) = \log_2 |Y| \tag{7}$$

achieved when all codes have equal probability.

7 Extensions and Generalizations

7.1 Dynamic Entropy Maps

For functions that change over time:

Definition 7.1 (Versioned Entropy Map)**.** *Maintain a sequence of seeds $(s_1, s_2, \ldots, s_t)$ where each s_i implements $\widetilde{f_i}$ for the function at time i.*

Updates require finding new seeds, but queries remain fast.

7.2 Continuous Domains

For continuous domains, we can use discretization:

Example 7.2 (Approximate Real Functions)**.** *To implement $\widetilde{f} : \mathbb{R} \to \mathbb{R}$:*

1. *Discretize domain and range to finite precision*
2. *Apply entropy map to discretized function*
3. *Observation error combines discretization and entropy map errors*

7.3 Quantum Entropy Maps

The framework extends naturally to quantum computation:

Definition 7.3 (Quantum Entropy Map)**.** *Uses quantum superposition to store multiple seeds simultaneously, enabling quantum queries of observed functions.*

8 Related Work

8.1 Connection to Existing Structures

Entropy maps generalize several well-known probabilistic data structures:

- **Bloom filters**: Entropy maps for set membership with specific codes
- **Cuckoo filters**: Entropy maps with additional deletion support
- **Minimal perfect hashing**: Entropy maps with zero error rate

8.2 Theoretical Foundations

The concept builds on:

- **Kolmogorov complexity**: Optimal representation of functions
- **Rate-distortion theory**: Fundamental space-accuracy trade-offs
- **Oblivious data structures**: Privacy through limited information leakage

9 Implementation Considerations

9.1 Hash Function Requirements

Critical properties for the hash function h:

1. **Uniformity**: Outputs are uniformly distributed
2. **Independence**: Hash values for different inputs are independent
3. **Efficiency**: Fast evaluation for practical use

9.2 Space Efficiency

The information-theoretic lower bound for implementing observed sets provides a fundamental limit on space efficiency:

Theorem 9.1 (Space Lower Bound)**.** *A data structure implementing $\widetilde{S}$ with false positive rate α and true positive rate τ requires at least:*

$$-\tau \log_2 \alpha \text{ bits per element} \tag{8}$$

This bound allows us to measure the absolute efficiency of implementations:

Definition 9.2 (Absolute Space Efficiency)**.** *For a data structure using B bits to store n elements:*

$$\eta = \frac{-n\tau \log_2 \alpha}{B} \tag{9}$$

where $\eta \in (0, 1]$ and $\eta = 1$ indicates optimal space usage.

Example 9.3 (Efficiency of Common Structures)**.**

- ***Bloom filters**: $\eta \approx 0.69$ (factor of $1/\ln 2$)*
- ***Entropy maps**: $\eta \to 1$ as code length increases*
- ***Perfect hash filters**: $\eta \approx 0.89$ for practical parameters*

Common choices: SHA-256, BLAKE3, or specialized hash functions for short outputs.

9.3 Code Assignment Strategies

Example 9.4 (Binary Tree Codes)**.** *Assign codes using a binary tree:*

```
True:  00, 01, 100, 101, 110, 111  (3/4 probability)
False: 11                           (1/4 probability)
```

Achieves false positive rate of 1/4 with average code length 1.81 bits.

9.4 Optimization Techniques

- **Parallel seed search**: Try multiple seeds simultaneously
- **Incremental construction**: Add elements one at a time
- **Approximate construction**: Allow a few errors for faster building

10 Experimental Evaluation

10.1 Construction Time vs. Space Trade-off

For set membership with n elements and false positive rate α:

- Space: $n \log_2(1/\alpha)$ bits (optimal)
- Construction time: $O(2^{n \log_2(1/\alpha)})$ (exponential)
- Query time: $O(1)$ (constant)

This confirms the fundamental trade-off: optimal space requires exponential construction time.

10.2 Comparison with Bloom Filters

11 Future Directions

11.1 Practical Constructions

The exponential construction time limits practical applications. Future work should explore:

- Approximation algorithms for seed finding
- Specialized constructions for common function classes
- Hardware acceleration for seed search

Property	Bloom Filter	Entropy Map
Space (bits)	$1.44n \log_2(1/\alpha)$	$n \log_2(1/\alpha)$
Construction	$O(n)$	$O(2^{n \log_2(1/\alpha)})$
Query	$O(k)$	$O(1)$
Oblivious	No	Yes

Table 1: Bloom filter vs. entropy map for n elements, FPR α

11.2 Theoretical Questions

Open problems include:

- Can we achieve subexponential construction for interesting function classes?
- What is the complexity of verifying an entropy map implements a given function?
- How do quantum algorithms change the construction complexity?

11.3 Applications to Machine Learning

Entropy maps could enable:

- Model compression through function approximation
- Privacy-preserving inference
- Uncertainty quantification through observation distributions

12 Conclusions

Entropy maps provide a principled approach to implementing Bernoulli maps—observed functions that approximate latent mathematical mappings. By using prefix-free codes and cryptographic hashing, we achieve:

- **Space optimality**: Using exactly the entropy of the observation distribution
- **Oblivious computation**: Stored representation reveals minimal information
- **Unified framework**: Generalizing Bloom filters, perfect hashing, and more

The key insight is that optimal space usage requires accepting observation errors—the latent/observed distinction is not a limitation but a fundamental requirement for efficiency.

While exponential construction time limits current practicality, entropy maps establish the theoretical foundations for space-optimal approximate computation. They demonstrate that the gap between latent mathematical functions and their computational observations can be bridged optimally through information-theoretic coding.

References